Analyses: Classification with MLP

03/01/2024

Description

Load Packages

If the libraries are not installed yet, you need to install them using, for example, the command: install.packages("ggplot2"). For the Hrate package this is different, since it comes from github. The devtools library needs to be installed, and then the install_github() function is used.

```
# install the latest version of neuralnet with bug fixes: devtools::install_github("bips-hb/neuralnet")
library(neuralnet)
library(sigmoid)
library(caret)
```

Load Data

Load data table with estimated feature values per sequence.

```
# load feature value estimations
estimations.df <- read.csv("~/Github/UpperPalCombinatorics/results/features/features_combined.csv")
head(estimations.df)
```

```
##     subcorpus       sequence num.char huni.chars hrate.chars  ttr.chars
## 1 Aurignacien ||| ////// _ V       12  1.8962406   1.5295563 0.41666667
## 2 Aurignacien          vvvvv        5  0.0000000   0.5711903 0.20000000
## 3 Aurignacien      vvvv vvvv        9  0.5032583   0.9060047 0.22222222
## 4 Aurignacien vvvvvvvvvvvvvv       14  0.0000000   0.6379550 0.07142857
## 5 Aurignacien            |||        3  0.0000000   0.4308271 0.33333333
## 6 Aurignacien           ||||        4  0.0000000   0.5070802 0.25000000
##    rm.chars
## 1 0.6363636
## 2 1.0000000
## 3 0.7500000
## 4 1.0000000
## 5 1.0000000
## 6 1.0000000
```

Choose subcorpora for binary classification.

```
# choose subcorpora to compare: "Aurignacien", "TeDDi", "Cuneiform (Uruk V)",
# "Cuneiform (Uruk IV)", "Cuneiform (Uruk III)".
subcorpus1 <- "Cuneiform (Uruk III)"
subcorpus2 <- "Cuneiform (Uruk V)"
selected <- c(subcorpus1, subcorpus2)
estimations.df <- estimations.df[estimations.df$subcorpus %in% selected, ]
```

Select relevant columns of the data frame, i.e. the measures to be included in classification and the ‘’subcorpus” column.

```
estimations.subset <- estimations.df[c("subcorpus",
                                       "huni.chars",
                                       "hrate.chars",
                                       "ttr.chars",
                                       "rm.chars"
                                       )]
```

Remove NAs (whole row)

```
estimations.subset <- na.omit(estimations.subset)
```

Center and scale the data

```
# first column is subcorpus column, the others are the features
# be aware that this needs to be changed in case of other table format
estimations.scaled <- cbind(estimations.subset["subcorpus"],
                            scale(estimations.subset[, c("huni.chars", "hrate.chars",
                                                         "ttr.chars", "rm.chars")]))
```

Create Training and Test Sets

```
# Generating seed
set.seed(1234)
# Randomly generating our training and test samples with a respective ratio of 2/3 and 1/3
datasample <- sample(2, nrow(estimations.scaled), replace = TRUE, prob = c(0.67, 0.33))
# Generate training set (select rows randomly by using datasample vector)
train <- estimations.scaled[datasample == 1, 1:ncol(estimations.scaled)]
# Generate test set
test <- estimations.scaled[datasample == 2, 1:ncol(estimations.scaled)]
```

Implement MLP classifier

This is partly based on code given at http://uc-r.github.io/ann_classification (last accessed 18.01.2023). The activation function (tanh), error function (SSE), and backpropagation algorithm (rprop+) are here chosen according to the best model performance reported in Bentz (2023).

```
# Generate list of hidden unit architectures (number of layers and units)
# Choose number of random architectures
arch.no <- 100
# initialize empty list of hidden unit architectures
hidden.list <- vector(mode = "list", length = arch.no)
set.seed(09012020)
# run loop to generate number of layers and hidden units randomly
for (i in 1:arch.no) {
  layer.no <- round(runif(1, min = 1, max = 4), 0)
 hidden.units <- round(runif(layer.no, min = 1, max = 5), 0)
  hidden.list[[i]] <- hidden.units
```

```r
}

# Manual list of hidden layer structures (for plotting and experimenting)
# hidden.list <- list(c(4,4))

# initialize dataframe to append results to
results.df <- data.frame(hidden.structure = character(0),
                         hidden.size = numeric(0), hidden.depth = numeric(0),
                         err.fct = character(0), act.fct.name = character(0),
                         algorithm = character(0), accuracy = numeric(0),
                         precision = numeric(0), recall = numeric(0),
                         f1 = numeric(0)
                         )

# Custom activation function
# Note that this function only works with err.fct = 'sse'
softplus <- function(x) log(1 + exp(x))

# set random seed for reproducibility
set.seed(123)
# counter
counter <- 0
# start time
start_time <- Sys.time()
for (hidden in hidden.list) {
  # Training
  # choose error function
  err.fct = 'sse' # options: 'sse', 'ce'
  # choose activation function
  act.fct = 'tanh' # options: 'logistic', 'tanh', softplus, relu,
  # beware: the last two options have to be written without quotes ''
  # note: softplus, relu, and tanh only work properly with sse as err.fct
  act.fct.name = 'tanh' # make sure to give name as character string (for later storage)
  # choose algorithm
  algorithm = 'rprop+' # options: 'rprop+', 'rprop-', 'backprop', 'sag', 'slr'
  # train classifier with neuralnet() function
  try({ # if the processing failes for a certain file, there will be no output for this file,
  # but the try() function allows the loop to keep running
  classifier.mlp <- neuralnet(subcorpus == subcorpus1 ~ .,
                     data = train,
                     hidden = hidden,
                     threshold = 0.1, # defaults to 0.01
                     rep = 1, # number of reps in which new initial values are used,
                     # (essentially the same as a for loop)
                     stepmax = 100000, # defaults to 100K
                     linear.output = FALSE,
                     algorithm = algorithm, # defaults to "rprop+",
                     # i.e. resilient backpropagation
                     # learningrate = 0.1,
                     err.fct = err.fct,
                     act.fct = act.fct,
                     likelihood = TRUE,
                     lifesign = 'minimal')
```

```
  classifier.mlp
  # results matrix (each column represents one repetition)
  classifier.mlp$result.matrix

  # Prediction
  # Get prediction using the predict() function
  mlp.predictions <- predict(classifier.mlp, test,
                         rep = which.min(classifier.mlp$result.matrix[1,]), # Predict response values
                         # based on the "best" repetition (epoche), i.e. the one with the lowest erro
                         all.units = FALSE)
  # assign a label according to the rule that the label is "modern writing"
  # if the prediction probability is >0.5, else assign "paleolithic".
  mlp.predictions.rd <- ifelse(mlp.predictions > 0.5, subcorpus1, subcorpus2)

  # Model Evaluation
  # creating a dataframe from known (true) test labels
  test.labels <- data.frame(test$subcorpus)
  # combining predicted and known classes
  class.comparison <- data.frame(mlp.predictions.rd, test.labels)
  # giving appropriate column names
  names(class.comparison) <- c("predicted", "observed")
  # inspecting our results table
  head(class.comparison)
  # get confusion matrix
  cm <- confusionMatrix(as.factor(class.comparison$predicted),
                    reference = as.factor(class.comparison$observed))
  #print(cm)
  # get precision, recall, and f1 from the output list of confusionMatrix()
  accuracy <- cm$overall['Accuracy']
  f1 <- cm[["byClass"]]["F1"]
  recall <- cm[["byClass"]]["Recall"]
  precision <- cm[["byClass"]]["Precision"]

  # unname and round resulting values
  accuracy <- round(unname(accuracy), 2)
  f1 <- round(unname(f1), 2)
  recall <- round(unname(recall), 2)
  precision <- round(unname(precision), 2)

  # append results to dataframe
  hidden.structure <- paste(unlist(hidden), collapse = ",")
  hidden.size <- sum(unlist(hidden))
  hidden.depth <- length(hidden)
  local.df <- data.frame(hidden.structure, hidden.size,
                       hidden.depth, err.fct, act.fct.name, algorithm,
                       accuracy, precision, recall, f1)
  results.df <- rbind(results.df, local.df)
  })
  counter <- counter + 1
  print(counter)
}
```

```
## [1] 1
## [1] 2
```

```
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 3
## [1] 4
## [1] 5
## [1] 6
## [1] 7
## [1] 8
## [1] 9
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 10
## [1] 11
## [1] 12
## [1] 13
## [1] 14
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 15
## [1] 16
## [1] 17
## [1] 18
## [1] 19
## [1] 20
## [1] 21
## [1] 22
## [1] 23
## [1] 24
## [1] 25
## [1] 26
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 27
## [1] 28
## [1] 29
## [1] 30
## [1] 31
## [1] 32
## [1] 33
## [1] 34
## [1] 35
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 36
## [1] 37
## [1] 38
## [1] 39
## [1] 40
## [1] 41
## [1] 42
## [1] 43
## [1] 44
## [1] 45
## [1] 46
```

```
## [1] 47
## [1] 48
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 49
## [1] 50
## [1] 51
## [1] 52
## [1] 53
## [1] 54
## [1] 55
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 56
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 57
## [1] 58
## [1] 59
## [1] 60
## [1] 61
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 62
## [1] 63
## [1] 64
## [1] 65
## [1] 66
## [1] 67
## [1] 68
## [1] 69
## [1] 70
## [1] 71
## [1] 72
## [1] 73
## [1] 74
## [1] 75
## [1] 76
## [1] 77
## [1] 78
## [1] 79
## [1] 80
## [1] 81
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 82
## [1] 83
## [1] 84
## [1] 85
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 86
## [1] 87
## [1] 88
```

```
## [1] 89
## [1] 90
## [1] 91
## [1] 92
## [1] 93
## [1] 94
## [1] 95
## [1] 96
## [1] 97
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 98
## [1] 99
## [1] 100
```

```
end_time <- Sys.time()
proc.time <- end_time - start_time
print(proc.time)
```

```
## Time difference of 7.17844 mins
```

Write to file.

```
write.csv(results.df, file = paste(c("~/Github/UpperPalCombinatorics/results/classifier/MLP/results_MLP_
                                             act.fct.name, "_",
                                             err.fct, "_",
                                             algorithm, "_",
                                             subcorpus1, "_",
                                             subcorpus2, "_",
                                             ".csv"
                                             ), collapse = ""),
                                     row.names = F)
```

Visualize the NN

Visualize the nn with the best weights after training.

```
#mlp.plot <- plot(classifier.mlp, rep = 'best', show.weights = F)
#mlp.plot
```

#References Bentz, Christian (2023). The Zipfian Challenge: Learning the statistical fingerprint of natural languages. CoNLL, Singapore. https://aclanthology.org/2023.conll-1.3/